Software Quality Journal (2024) 32:1287–1324
https://doi.org/10.1007/s11219-024-09686-0

RESEARCH

Exploring behaviours of RESTful APIs in an industrial setting

Stefan Karlsson[1,2] · Robbert Jongeling[2] · Adnan Čaušević[1] · Daniel Sundmark[2]

Accepted: 7 June 2024 / Published online: 3 July 2024
© The Author(s) 2024

Abstract
A common way of exposing functionality in contemporary systems is by providing a Web-API based on the REST API architectural guidelines. To describe REST APIs, the industry standard is currently OpenAPI-specifications. Test generation and fuzzing methods targeting OpenAPI-described REST APIs have been a very active research area in recent years. An open research challenge is to aid users in better understanding their API, in addition to finding faults and to cover all the code. In this paper, we address this challenge by proposing a set of behavioural properties, common to REST APIs, which are used to generate examples of behaviours that these APIs exhibit. These examples can be used both (i) to further the understanding of the API and (ii) as a source of automatic test cases. Our evaluation shows that our approach can generate examples deemed *relevant* for understanding the system and for a source of test generation by practitioners. In addition, we show that basing test generation on behavioural properties provides tests that are less dependent on the state of the system, while at the same time yielding a similar code coverage as state-of-the-art methods in REST API fuzzing in a given time limit.

Keywords Property-based testing · Examples · Automated testing · REST API test generation · OpenAPI

1 Introduction

Web technology is a common way for systems to expose software functionality to clients, often following the architectural style of REST (Fielding, 2000). Services exposing REST Application Programming Interfaces (APIs) are common both for publicly available

✉ Stefan Karlsson
stefan.l.karlsson@mdu.se; stefan.l.karlsson@se.abb.com

Robbert Jongeling
robbert.jongeling@mdu.se

Adnan Čaušević
adnan.causevic@se.abb.com

Daniel Sundmark
daniel.sundmark@mdu.se

[1] ABB AB, Västerås, Sweden

[2] Mälardalen University, Västerås, Sweden

services, such as those provided by Google,[1] Amazon[2] and Microsoft,[3] and for internal systems, in a microservice-based architecture (Fowler, 2016). To enable developers to use an API, some kind of description must be provided. Such descriptions can be provided in natural language, as written documentation, or in a more formally specified format. The *de facto* method of providing a specification—if any is provided—is by using OpenAPI-specifications.[4] An OpenAPI-specification describes the exposed operations, their parameters and responses. The adoption of OpenAPI specifications is widespread and increasing (Serbout et al., 2022).

Given the popularity of REST APIs, methods to aid practitioners assess the quality of REST APIs have been of large interest in the research literature, with many different testing methods proposed (Arcuri, 2019; Atlidakis et al., 2019, 2020a; Ed-douibi et al., 2018; Karlsson et al., 2020; Laranjeiro et al., 2021; Martin-Lopez et al., 2021; Viglianisi et al., 2020; Wu et al., 2022). The proposed methods cover a range of approaches, for example, search-based, property-based, and model-based techniques (Kim et al., 2022). A common factor of the proposed methods is the reliance on an OpenAPI-specification, describing the operations of the system-under-test (SUT). Thus, OpenAPI-specifications are central to the current state-of-the-art (SotA) in test generation and fuzzing (generating and executing pseudo-random inputs to find faults) of REST APIs.

The common evaluation metrics in REST API test generation are fault finding and code coverage. According to recent studies (Kim et al., 2022; Zhang & Arcuri, 2022), the currently best performing method—both in terms of code coverage (Kim et al., 2022; Zhang & Arcuri, 2022) and fault finding (Kim et al., 2022)—is EvoMaster (Arcuri, 2019). Fuzzing with EvoMaster produces on average about 50% code coverage, on a set of different SUTs (Kim et al., 2022; Zhang & Arcuri, 2022). However, the variability in coverage for different SUTs is quite high, ranging from about 15% in the worst case and 95% in the best case. These numbers are encouraging, for such a recent research area as REST API test generation. Regarding fault finding, a common measure of fault finding is to judge the API returning a 500 HTTP status-code (meaning "Internal Server Error") as a fault (Golmohammadi et al., 2023). However, aside from reaching high code coverage and fault finding, understanding how the API behaves is important and essential to delivering the required functionality of an API.

OpenAPI specifications include the operations, parameters, and responses of APIs. These specifications have enabled the automated generation of test cases that check the responses of the system with respect to the specified response or crashes. However, there is an opportunity to leverage automated generation based on OpenAPI specifications to provide information to users who want to use and understand the API. To gain this understanding, usage examples are an important resource (Aghajani et al., 2020; McLellan et al., 1998; Nykaza et al., 2002; Novick & Ward, 2006; Robillard, 2009; Robillard & DeLine, 2011; Shull et al., 2000; Sohan et al., 2017). In addition, relationships can exist between specified API operations. For example, the operations might have to be invoked in a specific order, or that one operation affects the output of another operation. If relationships between distinct API operations are unclear, discovering those can put a large burden on the user (Piccioni et al., 2013). Examples are of great value for understanding the

[1] https://cloud.google.com/
[2] https://aws.amazon.com/
[3] https://azure.microsoft.com/en-us/
[4] https://www.openapis.org/

behaviour of an API, thus, automatically generating such examples is valuable. In addition, if the generation can leverage already available artefacts—an OpenAPI specification for REST APIs—the required user effort is low.

A novelty of the approach proposed in this paper is to allow for automatic example generation for REST APIs, without the requirement of white-box information or a formal specification of the APIs behaviours. Our approach generates the examples through automated interactions with the system.

In this paper, we primarily target the challenge of generating relevant examples of RESTful API behaviours to aid users in better understanding the behaviours of the API and to leverage such examples for test generation. We define behaviour as what an API should or should not do. Using generated relevant examples for test generation addresses several aspects in REST API test generation, including (i) generating tests closer related to business logic,[5] (ii) reducing the time of test generation, and (iii) decreasing the dependency on a clean state of the SUT in test generation and execution. All three of these have been identified as open research challenges (Kim et al., 2022; Zhang et al., 2022). The approach we propose uses behavioural properties, common to REST APIs, to generate examples of behaviours. These examples are used as a source for test cases, and also as a source to further practitioners understanding of the API. Our approach, focusing on examples of behaviours complements approaches solely focusing on code coverage and fault finding—a very important purpose—which generate tests only furthering those goals. Consider an example of input validation, where an API operation *does not* put any constraints on the input, i.e., there is no input validation code to cover. To fulfil the criteria of code coverage, we only have to execute the operation once. However, such a test may not show the actual behaviour when looking at the produced test case; the *absence* of input validation. Focusing on behaviours opens up the possibility of generating tests for such scenarios, moving test generation closer to the actual business logic of REST APIs.

To evaluate our approach of using generated examples based on behavioural properties for REST API test-generation, we have extended the Property-based REST API test-generation method QuickREST (Karlsson et al., 2020). Our main claim with the proposed approach is that by using an example generation approach, users can *both* generate a source of understanding the SUT and a source for test generation—with comparable coverage to the SotA. We capture this claim in three research questions; **RQ1**: "How do practitioners perceive the relevance of the generated examples?", **RQ2**: "How does our behaviour-driven test-generation method compare to test-generation with EvoMaster, based on search-coverage?", and, **RQ3**: "How do search and execution coverage compare between the methods and what are any limiting factors?". In order to support this claim, we provide the results of two evaluations. First, we evaluate if our proposed approach provide practitioners with relevant examples of behaviours which are useful and aid in understanding the behaviour of the SUT (**RQ1**). We do so by performing focus group sessions with experienced industry practitioners. Secondly, we use QuickREST with our extension and compare to EvoMaster, which is the current SotA for test generation driven by code coverage and fault finding. We evaluate the search coverage of our behaviour-based method, i.e., the code covered during the search for examples or test cases, (**RQ2**), and also how the execution coverage of the resulting artefact behaves, in relation to the search coverage (**RQ3**). Using code coverage as an evaluation metric is useful since a strong correlation between coverage and fault finding has been shown when evaluating different methods of REST API test generation (Kim et al., 2022).

[5] https://en.wikipedia.org/wiki/Business_logic

Our approach produces a set of examples of behaviours that can be used both as a guide to understand the system, and as a test suite. Our evaluation shows that practitioners deem these generated examples as relevant and see multiple usage areas for them. At the same time, using the generated examples as automatic test cases yields a code coverage comparable to the SotA. In addition, test suites resulting from our approach are more robust to changes in the SUT state, making them more useful for regression testing, compared to test cases based on more specific expectations than general behaviours. In summary, the contribution we make with this paper is a novel approach of REST API test generation, based on a set of proposed common behaviours of REST APIs producing relevant examples. This approach produces tests intending to cover behaviours, closer to the business logic of a system and provide API usage examples, which in addition can also be used to further the understanding of the SUT.

The structure of this paper is as follows; in Section 2 we outline the key idea of the proposed approach. In Section 3 we introduce our approach in detail. This is followed by the evaluation of the approach in two parts. In Section 4, we present the evaluation of the approaches ability to generate relevant examples. The evaluation of test-generation is in Section 5. In Section 6 we discuss the results. The paper is then concluded with related work, Section 7, and finally conclusions, Section 9.

2 Key idea: generating examples of REST API behaviours

The key idea of our proposed approach is to use example generation as a source of understanding and test generation. The approach we propose generates examples of common REST API behaviours, such as managing the entities of the exposed system by creating, reading, and deleting them. We define these common behaviours as properties, for example, stating that a sequence of operations should incur a state-change in the system. By "examples", we refer to sequences of API calls that show a particular behaviour. In this paper, we generate such examples as sequences of REST API calls that show particular common REST API behaviours. Since the examples are sequences of API calls, they can be leveraged as tests themselves, or simply as a source of understanding the API, or as documentation for engineers/users of the API. In quality assurance and development activities, QA engineers as well as software developers can inspect the generated examples as an aid in understanding if the API delivers the sought behaviours, conforming to the requirements. Figure 1 shows an overview of possible ways of working with generated examples.

To aid in the understanding of an API, generated examples should only include those API operations needed to show the exemplified behaviours, in other words, the examples must be *relevant*. Relevant examples should include only those API operations that are important to the behaviour that the interaction exemplifies (Gerdes et al., 2018; Karlsson et al., 2024; Robillard & DeLine, 2011). In addition, Gerdes et al. (2018) found that code coverage is not a strong heuristic for the relevance of generated examples. Thus, a key idea of our approach is to focus generation on finding behaviours, rather than metrics such as fault finding and code coverage.

Our proposed approach builds upon a general API example generation approach, introduced by Karlsson et al. (2024). We adapt the general approach and put it in the context of REST API test generation. By doing so, we can generate examples of common REST API behaviours which can be used as test cases. In particular, we generate specific examples for REST APIs that can be both used as a source of information on the API's behaviour

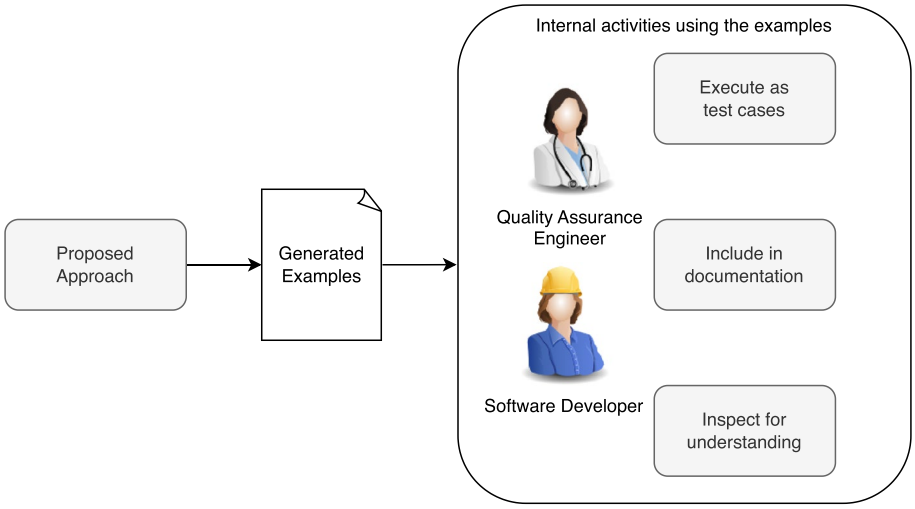


Fig. 1 Overview of how the generated examples can be used to support internal development activities

and as a source for test generation. The approach can be used as a complement to existing fuzzing methods, producing a broader range of test cases closer to the business logic of the application.

To generate examples of REST APIs behaviour, we define and use *behavioural properties* common to RESTful APIs. These behaviours are based on the CRUD (*Create, Read, Update, Delete*) idiom, typical for APIs managing entities. We use an OpenAPI specification to know which operations the API provides. We then leverage test generation to generate trial sequences, execute those on the system under test, and judge if the sequence matches any of the defined behavioural properties. This is done in a black-box fashion, since the only required input to the approach is the OpenAPI specification.

OpenAPI specifications are widespread, at least in open repositories such as GitHub (Serbout et al., 2022; Serbout & Pautasso, 2024), SwaggerHub, and APIs.Guru (Serbout & Pautasso, 2024). In our experience, if a REST API has a specification, it will be an OpenAPI specification. However, based on data from open repositories, it is difficult to assess how prevalent its use is in industry, and we have seen it both mandated for teams as well as teams not creating specifications for their APIs at all. For our approach, we believe the need for an OpenAPI specification as input will not pose a limiting factor. Even if not yet created, the integration of automatic generation of OpenAPI specifications in mainstream web-frameworks, such as ASP.NET[6] and the Java Spring framework,[7] lower the threshold for adoption. Moreover, due to OpenAPI specifications being lightweight, it is a reasonable option to manually create one if an automatic option is not available.

Specifically targeting behaviours of RESTful APIs provides an opportunity to leverage information specific to this API domain in the example generation process. Such information is the common CRUD idioms of RESTful APIs, and the expectations of the purpose of the HTTP methods, such as GET, POST, and DELETE.

[6] https://github.com/domaindrivendev/Swashbuckle.AspNetCore
[7] https://springdoc.org/

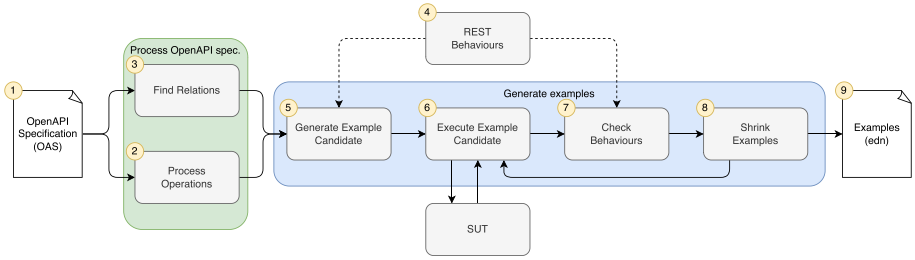


Fig. 2 Overview of the example generation process

In using our example generating approach, we can produce test cases that cover behaviour, but that do not necessarily contribute to other metrics, such as code coverage.

3 Proposed approach

In this section, we describe the details of our proposed approach. We explain how *behavioural properties* are used to generate examples as test-cases for REST APIs. In doing so, we present the overarching algorithm and the specific properties formulated. Further, we present our solution for how to relate different operations in an OpenAPI specification (OAS).

Figure 2 shows an overview of the approach. On a high-level, the input to the process is an OAS (1). We first process the OAS to know which operations are available (2) and how they relate (3). This information is then used in conjunction with defined REST behaviours (4) to generate examples. Generated example sequences (5) are executed on the SUT (6) to observe how the SUT behaves in context of the example sequence (7). The output is the shrunk (8) sequences of operations that passed a check of a defined behaviour (9). The following sections will describe these parts in more detail.

A user of this approach can produce test cases that exemplify behaviours of the SUT that the user might expect, or be surprised by. In some cases the SUT might not expose some of the behaviours the approach can find. In those cases, a lack of examples is the expected output—when the SUT does not express the given behaviour, no such examples should be produced. This method can not only be used for quality assurance purposes. The method is also applicable in cases where a user wants to learn more about an API—where examples may be beneficial (Robillard & DeLine, 2011). Perhaps a developer is using a 3rd party API which lacks documentation. By generating behavioural-based test cases, the user can find out how this particular system behaves with respect to the behavioural properties defined.

3.1 Defining behaviours

We use the term *behaviour* in this paper. We use the term to focus on what an API should or should not do and to distinguish from *faults*. A correct program will be free from faults, but it will have behaviours—behaviours are the reason a program exists. We use the term behaviour as describing a sequence of operations, with parameters if needed, that when executed on the SUT produces an observable nominal effect. The observable effect can

be of different kinds. For example, one effect of executing a sequence of operations could be that we get an equal response from each operation, or the responses are different, or the sequence affects the response of another operation, etc. In a black-box scenario, we can only observe what the SUT returns. In the case of REST APIs, this is the status code and potentially data contained in the response.

A definition of a behavioural property serves as a general model in which a REST API specific behaviour is expressed. We summarize the description of a *Behavioural Property* in Definition 1.

Definition 1 (Behavioural Property). A *Behavioural Property* is a property, $B = \langle C(O), Q \rangle$, defining a tuple containing a predicate C, based on the observations, O, determining if the effects of executing a sequence of operations, generated from Q, conforms to a defined pattern.

Given our definition of behaviour in mind, the question is then; what sequences of operations should be generated and what specific effect-patterns to check?

3.2 Definition of REST API-specific behavioural properties

The knowledge that can be derived from a REST API is, for example, the meaning of the HTTP-methods used to invoke the API. An example of such knowledge would be an operation declared as an HTTP DELETE method—it can be derived that the intent of the operation is to delete an entity. Thus, when stating behaviours we can leverage that, for example, a GET-operation is expected to query entities and that DELETE-operations are expected to delete entities. This knowledge can be used both in the generation of sequences of operations to execute and in the conformance predicate checks of the observed effects of the execution of a sequence. In sequence generation, we can use this knowledge to set different probabilities of types of operations in the generated sequence—for example, for some behaviours it is more likely that a delete operation follows a create operation—or to select specific types of operations in certain positions in the sequence. For example, if we want to generate tests for a behaviour of successful deletion, it is pointless to generate a DELETE-operation prior to a POST-operation (create) of an entity. In the conformance checks of the observed effects, we can leverage REST-specific knowledge, such as a GET-operation (query) should not change the state of the SUT, while a successful DELETE-operation should.

We define behavioural properties for REST APIs as:

Definition 2 (RESTful API Behavioural Property). A RESTful API Behavioural Property, $B = \langle C(O), Q \rangle$, is a tuple including a predicate, C, which judge if a sequence of observations, O conforms to an expected RESTful API behaviour, a generator Q which generates execution sequences of HTTP-methods of a REST API, according to some selection rule.

We consider an example of using this definition in which the generator Q is defined to generate sequences of (i) first a GET-operation, (ii) any number of POST-operations, and (iii) the same GET-operation as started the sequence. When a sequence of API-calls generated by Q is executed, the results returned by the API, the operation response and the status-code, are contained in O. After executing the API-call sequence, O is judged by the boolean function C. In this example, C will check if the observation O_1 is not equal to observation O_n, i.e., the observation of the first GET in the sequence and the last GET in

the sequence should be different. If C yields a positive result the observations is judged to conform to the behaviour, otherwise not.

There is, probably, an infinite set of possible behaviours of an API. Therefore, in practice, we must restrict ourselves to a finite set. A common idiom for REST APIs is to provide operations to manipulate entities, *Create-, Read-, Update-, and Delete*-operations (CRUD). Thus, targeting CRUD would be a reasonable set of REST API behaviours to specify, which is what we do in this work. The specifics of these selected behaviours are expanded in Section 3.5.

3.3 Exploration with property-based testing

In this paper, we propose a method to explore the behavioural properties of a REST API by generating examples. As defined in Definition 1, we do this with the components of a sequence generator and a predicate based on the observations of executing the sequence. These components fit well into Property-based testing (PBT) (Claessen & Hughes, 2000). PBT libraries provide generators for basic types of values and some execution layer to run a specific number of trials when evaluating a property by random generation. However, these libraries typically only provide basic generators—such as for basic data types as strings, integers, etc.—and combinators, and the challenge of creating compositions of domain-specific generators and predicates are left to the user of the library.

PBT has been used in the context of REST APIs. Prior work by Karlsson et al. has shown that PBT is a method useful for automatic fault finding of OpenAPI-described RESTful APIs (Karlsson et al., 2020). The focus of the work in Karlsson et al. (2020) was on fault finding, the proposed properties used were aimed at finding crashes (500 status codes in the case of REST APIs), and the conformance to the OpenAPI-specification of the actual responses of the SUT. In this work we take a different approach. While we still keep properties of fault finding—those are certainly of great importance—we also use behaviour-based properties, as described. We focus on, in addition to fault finding, generating tests to show behaviours (or the lack thereof) expected of a RESTful API. The main consequences of this is that the generation of operation sequences will depend on what behaviour we are trying to generate examples of. Moreover, the predicated checks do not only consider faults, but what defines the sought behaviour.

3.4 Example-generation algorithm

In accordance with Definition 2, a behaviour consists of a check-function, C, over the observations, O, and an operation sequence generator function, Q. Thus, the first step of the exploration approach is to create those components based on the specific behaviour sought and the given API specification.

Referring to Algorithm 1, the three inputs to our exploration approach (L1) are the OAS, S, providing a schema of the operations, parameters, and responses; the specific behaviour, B, we are seeking; and the number of iterations N to search for an example before giving up.

Algorithm 1 Example Generation

```
1:  function EXPLORE(B, S, N)
2:      os ← QueryOperations(S)
3:      rs ← BuildSchemaGraph(S)
4:      B_Q ← BuildOperationsGenerator(B, os, rs)
5:      B_C ← BuildBehviourCheck(B, os)
6:      Example ← CheckProperty(B, N)
7:  end function

8:  function CHECKPROPERTY(B, N)
9:      for n ∈ 1..N do
10:         (result, E) ← CheckBehaviour(B_Q, B_C)
11:         if result then Shrink(E)
12:         else if n = N then return NoExampleFound
13:         else continue
14:         end if
15:     end for
16: end function

17: function CHECKBEHAVIOUR(Q, C)
18:     E ← Q_GenerateOperationsSequence()
19:     for e ∈ E do
20:         o ← ExecuteOperation(e)
21:         O ← O ∪ ProcessObservation(o)
22:     end for
23:     O_final ← ProcessObservations(O)
24: return (C_Check(E,O,O_final), E)
25: end function
```

We start by querying the operations of the OAS (L2) and build a graph of how the operations relate (L3). The generator for the specific behaviour, B_Q, is then created based on the available operations in the OAS and their relations (L4), in accordance with the sought behaviour. For example, if we seek a "delete" behaviour, we want the generator created, B_Q, to produce operation sequences with probable create operations of an entity (POST) before any deletion of the entity (DELETE). Which specific create and delete operations that can be generated as a trial example is based on how the operations, *os*, relate to each other, *rs*. The details of how relations are found are described further in Section 3.6. The construction of the check function, B_C, also depends on the behaviour we seek. Using the same "delete" example as above, the check function created for such behaviour might verify that the observed behaviour first contains an entity, then later in the sequence of observed responses from the SUT, no longer included the same entity. Hence, the entity has been deleted and the sequence would be an example conforming to the sought behaviour. With the parts required for the exploration assembled, the exploration process is started, with a maximum of *N* trials (L6). This will produce an *Example*, or in the case of reaching the trial limit, a marker for *NoExampleFound*.

It is worth noting, given the approach of our method, a *NoExampleFound* result for a given behaviour is not necessarily a bad thing. It can be the exact output we need to indicate that the SUT behave as we expect! For example, imagine a "create" operation allowing the creation of entities with the same parameters. If we seek examples of the behaviour where providing the same input twice results in a rejection of the second invocation, we will not find any examples—if the input validation is correct. Hence, not finding an example indicates expected behaviour, and found examples are potential bugs.

Considering the inner details of the algorithm (L8-25). We make N trials (L9) to find an example of the behaviour. Each trial of checking for a behaviour starts by generating a sequence of operations to execute (L18), E, using the previously created generator (created in L4) and relations (created in L3). Thus, each trial is a new sequence. Each operation in the sequence is executed on the SUT (L20) and the response of the execution is added to the set of observed responses. Before the observation is added, there is a possibility to process it (L21). An example of such processing is that we might only be interested in parts of the response, such as the body and not headers containing meta-data. The final result of the check for behaviour is boolean (returned in L24 as part of the result of the call in L10), whether the sequence of execution in combination with the observations corresponds to the sought behaviour. If we do get a positive example of the behaviour (L11), we try to shrink the execution sequence to an example as short as possible that still exhibits the behaviour.

In summary, the input to the exploration method is the sought behaviour and the OAS, the output is either a shrunken example exhibiting the behaviour or *NoExampleFound*.

3.5 Proposed REST API behavioural properties

When generating examples of general behaviours, Karlsson et al. defines and uses a set of *meta-properties* (Karlsson et al., 2024). Meta-properties define general behaviours, such as detecting a state-change. The behaviours we propose in this paper are specific to REST APIs. In doing so, we instantiate general meta-properties and specialise them given our context.

In this first set of proposed REST API behavioural properties, we limit the set of behaviours to CRUD-based operations. In doing so, we take advantage of expectations around the HTTP verbs GET ("read"), POST ("create"), PUT/POST ("update"), and DELETE ("delete"). Recall from Definition 2, a Behaviour is a combination of a Check function and a generator of HTTP-methods. When we define a specific behaviour we then consider both the generation of reasonable trials and the check on them, these components are not separate. For example, if the behaviour we are seeking is deletion, we can, by construction, make sure the generator used will always include DELETE operations in the generated sequences. Any other sequences are not relevant to use as trials.

The set of initial behaviours we define and evaluate are:

- **B1 - Equal response, same operation** - This behaviour generates a sequence of two invocations of the same operation with the same parameters. The range of examples this behaviour can produce is, for example, POST operations lacking input validation/restrictions, examples of GET operations reading the state of the SUT, or DELETE on an entity that does not exist, etc. Figure 3 shows two examples generated by this behaviour. The first example adds a product with the same name twice, i.e., enforcing unique names is not the current behaviour of this SUT. This example might confirm or surprise a developer/tester's perception of the SUT. The second example shows a GET operation returning the same response given the same parameters, and no other opera-

```
{"addProduct"
  {:operation-sequence
   [{:operation "addProduct",
     :parameters
     [{:name "productName", :value {"productName" "0"}}]}
    {:operation "addProduct",
     :parameters
     [{:name "productName", :value {"productName" "0"}}]}]
  }
}

{"getAllProducts"
  {:operation-sequence
   [{:operation "getAllProducts", :parameters []}
    {:operation "getAllProducts", :parameters []}]
  }
}
```

Fig. 3 Examples generated from behaviour B1

tion executed in between. Not finding such examples for GET operations would be a cause for further investigation.

- **B2 - Different response, same operation** - This behaviour generates a sequence of two invocations of the same operation, just as B1. However, the check of this behaviour expects the responses to be *different*. The examples produced can be thought of as inverted from B1. For example, if the SUT constrained the "productName" from the B1 example to be unique, it would show as an example of this behaviour. The first response would have a successful status-code (such as 201) and the second response would be a rejection (400, indicating a client error). Any examples of GET operations generated by this behaviour would indicate a deviation from how a user would expect a typical CRUD API to behave.

The defined behaviours B1 and B2 are simple; the sequence to test only consists of two operations. This is a trade-off between the complexity of behaviours possible to find and the duration of the search. A user making a change to input validation can choose to execute these behavioural properties, to get fast results on the behaviour currently under development. Still, while being simple, B1 and B2 have the ability to find several of the behaviours we expect of a CRUD API, as described, but are unable to find behaviours such as a successful deletion of a created entity. To get more complex examples, where the sequence of operations is not as constrained, we define two more behaviours. In the definitions, we categorize the HTTP-methods POST, DELETE, and PUT as potentially state-changing.

- **B3 - A sequence of state-changing operations changing the response of a GET** - Sequences generated by this behaviour always start and end with the same GET operation. Thus, there will be one exploration for every available GET operation in the OAS. By always starting and ending with a GET operation, changes in the state of the SUT can be detected. Between the GETs, there is a sequence of potential state-changing operations. When generating parameters for these operations, in addition to generating random values, an operation can reuse parameters from a previous operation in the sequence. In this way, we can perform multiple operations on the same entity, or use a previous entity as an argument to another operation, reach-

```
{:operation-sequence
   [{:operation "getConfigurationsForProduct",
     :parameters [{:name "productName",
     :value {"productName" "0"}}]}
    {:operation "addProduct",
     :parameters [{:name "productName",
     :value {"productName" "0"}}]}
    {:operation "addConfiguration",
     :parameters
     [{:name "productName", :value {"productName" "0"}}
      {:name "configurationName",
       :value {"configurationName" "0"}}]}
    {:operation "getConfigurationsForProduct",
     :parameters
     [{:name "productName", :value {"productName" "0"}}]}
     ]}
```

Fig. 4 Example generated by behaviour B3 with multiple operations on the same entity. A configuration is added to an added product in "features-service" case-study

ing further into the state of the SUT. Figure 8 shows an example generated by this behaviour. The sequence generated in the example, considering the HTTP methods, has the structure of GET-POST-GET. The GET-operations are the same, "getAllProducts", and the POST-operation in between, "addProduct", causes the responses of the first and last invocation of the GET-operations to differ. Figure 4 shows an example where an operation, "addConfiguration", uses an entity created earlier in the sequence of the example. The B3 behaviour has the ability to produce examples of typical "Read" and "Create" behaviour, as shown in the generated example.

- **B4 - A sequence of state-changing operations NOT changing the response of a GET** - This behaviour aims to generate examples where first a change is induced on the state of the SUT, which is then "cancelled out", typically by DELETE operations. Such an example would capture both "create" and "delete" behaviour in relation to a "read" operation. As with B3, we need to base the operations around GETs to observe any changes in the state of the SUT. The generator for this behaviour will generate sequences with GET operations before and after any sequence of at least two operations selected from the POST and DELETE operations in the OAS. As was also the case with B3, the relations between operations in the OAS are leveraged in the selection of operations, to avoid uninteresting sequences. As hinted in the title of this behaviour, the aim of the check for the behaviour is that we start and end with the same response of the specific GET. A typical example generated from this behaviour would be a GET-POST-DELETE-GET sequence. This means that the state was first read, an entity was created, the same entity was deleted, and we are back getting an equal response as we started with. Figure 5 shows an example from one of the case studies of this behaviour. The example includes a GET operation between the POST and DELETE, to inform the user of the state change. This GET could optionally be removed based on the intent of communication of the example. This would fit well into a parsing layer for different presentation options and is orthogonal to the method of producing the example.

```
{"getProductByName"
  {:operation-sequence
   [{:operation "getProductByName",
     :parameters [{:name "productName",
     :value {"productName" "B"}}]}
    {:operation "addProduct",
     :parameters [{:name "productName",
     :value {"productName" "B"}}]}
    {:operation "getProductByName",
     :parameters [{:name "productName",
     :value {"productName" "B"}}]}
    {:operation "deleteProductByName",
     :parameters [{:name "productName",
     :value {"productName" "B"}}]}
    {:operation "getProductByName",
     :parameters
     [{:name "productName",
     :value {"productName" "B"}}]}]}}
```

Fig. 5 Example generated by behaviour B4 in "features-service" case-study

The original QuickREST sought to find examples of operations and inputs which produced 500-status code results, i.e., crashes (Karlsson et al., 2020). We have included this ability as a "fuzzing" behaviour, in case a user wants to perform fuzzing instead of searching for the defined REST API behaviours.

In this section, we introduced the specific behaviours defined as the first set of behaviours to use in example generation for REST APIs. This is not an exhaustive list by any means, and the general approach presented is open to extensions to other behaviours in accordance with Definition 2, by defining other generators and checks for behaviours. When the example generation algorithm, Algorithm 1, yields a *NoExampleFound* result, the result is in the context of behaviours used in the search. In addition, as the approach is based on test generation (as described in Section 3.3) and not a formal proof system, the same limitation as for testing in general applies. As famously pointed out by Dijkstra: *"Testing shows the presence, not the absence of bugs"*. Thus, the approach cannot guarantee that an example does not exist given a negative result.

3.6 Type-based relation finding

One of the inputs to our method is an OpenAPI specification (OAS). The OAS describes the available API operations, their inputs and outputs, and any potential types used. However, the OAS does not specify relations and constraints between the operations and their inputs and outputs. This fact results in a common challenge of REST APIs test generation, finding dependencies among operations (Kim et al., 2022). Solving this challenge is required to generate operations in meaningful sequences, where one operation might depend on another.

When exploring different sequences of operations, in a stateful API, relations between operations are relevant. Such relations might be in the order the operations are executed, and also between the parameters and responses of the operations. For example, a value from a response of one operation might be used as the parameter of another operation. Given the ability to find relations, we are able to generate sequences of operations that perform operations on the same SUT entities. As an example, if we first create a person with

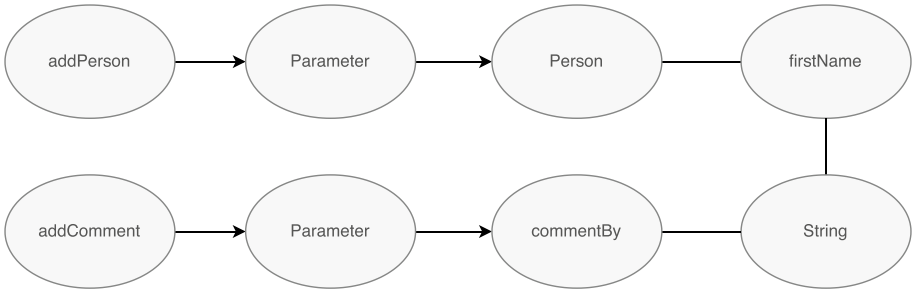


Fig. 6 Example of a type-based graph relation. Note the use of both directed and bidirectional relations

operation *A*, we might want to refer back to the same person when we delete the entity in operation *B*. Hence, operations A and B are related via their parameters.

In this paper, we use a graph-based approach to find relations, similar to what other test generation approaches do (Viglianisi et al., 2020; Zhang et al., 2021). However, we base our relation graph on the *types* of the operation parameters and responses in the OAS. The nodes in the graph are thus the types and the edges are relations between the types. Building a graph of the relations has been done before, but the relations have included the names of operations and parameters, and their URL resource relations (Viglianisi et al., 2020; Zhang et al., 2021). We only consider the types, down to scalar types, as, for example, strings. By building a type-based graph we can find relations that are not resource or name related, i.e., the operations might not share a common URL in the OAS. For example, imagine an API operation "addPerson" which takes as an input a "Person"-object with a field of "firstName". The first-name is marked as a "string" in the OAS. Another operation, "addComment" has a parameter called "commentBy" of type string. The operations do not share anything in the OAS, one has a parameter of "Person" and one of a plain string. However, with a type-based graph, there will be a path between these operations parameters, via the common string type. Hence, we can find the relationship when generating examples, between the "firstName" of a "Person" and the "commentBy" parameter. The example show us that there is a type-based connection between the "addPerson" and "addComments" operation, with the path shown in Fig. 6. We use directions on the relations to constrain the search. A node representing an operation type may relate to several different parameters. However, when searching for a candidate parameter, we only want to consider those that connect to the same basic type. In the example in Fig. 6, if searching for a candidate to use as a value for "commentBy", we want the search to end at "Person" (connected via a basic type, "string"), and not to continue to any other parameters of "addPerson" that do not connect to the same basic type. However, as with all solutions, there are trade-offs. With this method, compared to URL-based relation-graphs, we can find relations even if there are no shared URL or names in the operations, parameters and responses. The potential downside is that the graph search-space is larger since the graph is on a lower level, i.e., containing more nodes and relations. In the context of generating examples, the solution to the relation problem is just a means to be able to generate good examples. Thus, we do not make any claims or evaluate how this approach compare to other proposed relation finding approaches.

How the OAS is structured plays a part in how large the search-space will be. Two different ways of defining the types of parameters and responses in an OAS are either in-line, together with the operation, or as a reference to a type defined in the "Definitions" part

of the specification. For example, we can in the same part as we state that an operation takes a parameter state that it consists of an "age" of int and a "name" of string, or make a reference to a "Person" type with the same fields, expressed in the definitions of the specification. Thus, we get two different versions of types; nominal (by name) or structural (by similar structure). The nominal case means that all operations that use a reference to "Person" mean the same thing, whereas if two operations locally define the same structure of a type, we can not be sure it is the same thing. For example, two local definitions of "age" and "name" could both be referring to persons, i.e. related, or one is referring to cars, i.e. unrelated. Consequently, it is more costly to find potential relations in the structural case than in the nominal case. With named types, we know the type is shared.

We consider the distance when navigating the graph to find relations, i.e., the closer two parameters or responses are, the more likely they are deemed to concern the same domain entity. Each part of a structural type will be a node. Following the previous example, there will be 4 nodes [*firstName* : *string*] → [*string*] and [*age* : *int*] → [*int*]. If another parameter shares part of its structure then it will point to [*firstName* : *string*], if not it might point to [*string*] if it has a string parameter. The first case is thus deemed as a closer relation than the second case. Two parameters with relation to the same nominal or structural type will thus be more closely related than being related via a scalar type.

In summary, from the types defined in the given OAS, we create a graph based on these types. The graph also connects the operations, parameters, and responses to these types. We support both types declared in the OAS as nominal, i.e., references to named types, or structural, i.e., inlined definitions in the response/parameter declaration. Leveraging the paths found in this graph, we can test for probable relations when generating example trials.

3.7 Shrinking examples

To maximize the information density in the generated examples presented to a user, they should be minimal. The example generation process generates trial sequences and checks those for conformance of the sought behaviour, as described. Such a generated example might conform to a behaviour, but might still not be *minimal*. A minimal example would be an example only including the required operations to achieve the behaviour. For example, the generated example in Fig. 8 shows a GET-POST-GET sequence and is a minimal example to show the behaviour of creating an entity that affects the specific GETs. However, the same conformance to the behaviour would be reached if an arbitrary number of additional POSTs were inserted in the sequence. Inserting such redundant operations is not helpful to a user, it only adds noise and makes the example harder to read.

To generate minimal examples, we use *shrinking* of any trial examples that conform to a behaviour. The expanded tool, QuickREST, is based on property-based testing (Karlsson et al., 2020). Shrinking, i.e., producing minimal failing examples is a typical part of PBT (Claessen & Hughes, 2000). However, as with generators, the included shrinking algorithms are not domain aware and too naive for our use-case. The shrinking algorithm we use in our method tries to shrink the parameter values used when executing an operation, and also tries to shrink the sequence of operations. In the shrinking of the sequence our implementation respect relations. This means that if an operation refers to a value from an operation prior to the sequence, that dependency is not shrunk away. Dependent operations must all be removed or included in a trial of a shorter sequence. This strategy avoids shrinking trials with examples with broken references among operations. For example, in a sequence of GET-POST(x)-GET-GET-DELETE(x)-GET, we can shrink

away the redundant GETs and the DELETE—no operation has a dependency to DELETE later in the sequence. However, we do not shrink away the POST(x), since the DELETE operation has a reference, and operates on the same value, "x". This approach to shrinking generated examples—respecting relations—is similar to the approach proposed in the general example generation approach, proposed by Karlsson et al. (2024). But as with other parts of our proposed approach, we instantiate the general approach with the specialised knowledge of dealing with REST APIs. For example, we do not want to shrink away the GET-operations for behaviours which depend on the return value of the GETs to evaluate the conformance of the example.

Another challenge in providing minimal examples is the continuous change of the state in the system, such as entities being added and deleted in a database. In our approach, we do not expect or require the user to provide any reset functionality. The problem regarding state and shrinking is that, due to changes in state, an example that failed a behaviour conformance check might succeed in the next shrinking trial, and vice versa. The consequence of this is that our shrinking algorithm can not always produce the smallest example, some "noise operations" can be left, due to the SUT state changing during the shrinking process. The positive side of this trade-off is that resetting the state in this kind of system-level testing, as REST API testing often is, is costly, a cost we do not need to pay, for each trial.

3.8 The resulting artefact

The resulting artefact of running a test-generation or fuzzing tool is important to consider. The process of searching for test cases can be a time and resource-consuming process, while on the other hand executing the actual resulting artefact, such as a test suite, can be considerably less resource-consuming. As an example, the SotA method EvoMaster is recommended to run between 1 and 24 h while searching for a test suite with high coverage and fault finding.[8] The result is a test suite, in a configurable output format such as JUnit-format. Running the resulting suite can typically be executed in seconds or minutes, a large contrast to the time it might have taken to generate the suite. In addition, the resulting suite can be used in regression testing, complemented by humans, stored in source control, etc. Reasons such as these make it important for test generation tools to have an executable output, which several of the current SotA tools lack (Zhang & Arcuri, 2022).

However, there are downsides to producing a generated test suite in a test-framework format, such as, for example, JUnit. To execute such tests you now have a dependency on the test-framework used. As a user, you need to set up such a project to be able to run the artefact. This potential problem is amplified in a black-box scenario, where the technology of the SUT is not shared with the test suite, for example, the SUT might be written in PHP while the generated tests are in Java. You now need to understand how to execute tests expressed in another technology. Choosing to output the tests in the format of a common framework might make it easy to execute them—if the organization is familiar with the technology—and since the output then must be formatted as source code, there is a risk of readability issues.

The last point we want to highlight is also related to a source-code-based artefact. If instead of producing source-code, an approach provides its output in a common and

[8] https://github.com/EMResearch/EvoMaster

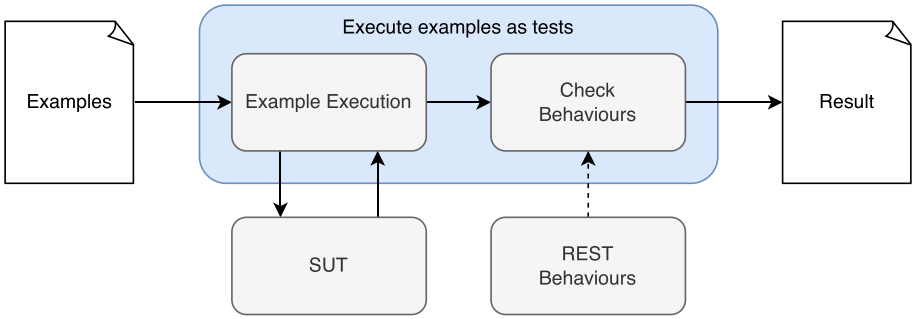


Fig. 7 Overview of the execution of examples as tests

well-defined data-structure format such as JSON,[9] it will allow users to further use the output. For example, if the user would like to produce an HTML-page with documentation based on the outcomes of the test-generation, making such programs is simplified if the test-generation artefact is straightforward to parse.

We acknowledge the importance of an executable artefact, as enumerated above, and also try to address the other mentioned issues. Our extension to QuickREST, which generates examples of behavioural properties, is able to execute its own output. As long as you have the tool, you can execute the generated tests without other dependencies. This is enabled by using a sub-part of the example generation process—as is shown in Fig. 7, instead of generating an example candidate, we execute and check an existing found example. The format of the output is a list of the operations in the example and any parameters, as in Fig. 8. The example is concise, helping readability, and regular, helping any further processing. Further studies are needed to evaluate which kind of output developers and testers prefer and in which scenarios. However, readability has been identified as an important challenge in REST API test generation (Zhang et al., 2022), and with our method, we provide an alternative to the test-framework-based approach. Figure 9 shows a longer generated example where an entity, a "product" in this case, is both created and deleted. Both examples in Figs. 8 and 9 are output from our case-study on the "feature-service" SUT.

4 Evaluation: relevance

The goal of the approach presented in this paper is to generate examples that can be used to advance the understanding of the SUT and to be used as test cases. We evaluate these two different aspects separately. In this section we present the evaluation with regards to generating *relevant* examples.

In order to evaluate if the examples generated by our proposed approach are *relevant*, we have conducted focus group sessions with industry practitioners. Focus groups are valuable to get early feedback on an approach, before investing further in the idea (Kontio et al., 2004). In addition, the openness of the evaluation method, provides an opportunity to capture a broad spectrum of feedback from practitioners (Kontio et al., 2004).

[9] https://www.ecma-international.org/publications-and-standards/standards/ecma-404/

```
{"getAllProducts"
  {:operation-sequence
   [{:operation "getAllProducts", :parameters []}
    {:operation "addProduct",
     :parameters [{:name "productName",
                   :value {"productName" "0"}}]}
    {:operation "getAllProducts", :parameters []}]}}
```

Fig. 8 Generated state-changing example

In this section we evaluate the research question of; **RQ1**: How do practitioners perceive the relevance of the generated examples?

4.1 Focus group and questionnaire setup

In order to understand if practitioners deem the generated examples as relevant and helpful, we performed two focus group sessions with two different agile teams at our industry partner. The two teams are development teams producing features for a digitalization platform for factory automation systems. REST APIs are one of the main types of APIs used in this product.

15 practitioners in total participated in the sessions, including the two teams with the addition of some cross-team roles. The session with the first team included 8 participants, and the session with the second team included 7 participants. The largest role group in the sessions was Software Engineers (10). The groups also included Product Owners (2), Quality Assurance Engineers (2), and a DevOps engineer (1). Performing the sessions with two complete agile teams—including supporting roles—gave us diversity in roles and experiences. The number of years of professional experience was also diverse among the participants, ranging from 2 to 24 years with an average of 13.2 years, based on questionnaire data.

The reported experience with REST APIs by the participants was 2.9—"Moderate"—in the quantitative data shown in Table 1. At a glance this number might seem low, considering that REST APIs are an important part of the deliverables of these teams. However, we make the following observations; 1) the diversity in roles, a product owner can have a good understanding of REST APIs on a product level (the product capabilities they deliver) but might feel a lack of technical depth when compared to software engineers with many years of experience. 2) The experience assessment is relative, i.e., an engineer with 2 years of

```
{"getAllProducts"
  {:operation-sequence
   [{:operation "getAllProducts", :parameters []}
    {:operation "addProduct",
     :parameters [{:name "productName",
                   :value {"productName" "A"}}]}
    {:operation "getAllProducts", :parameters []}
    {:operation "deleteProductByName",
     :parameters [{:name "productName",
                   :value {"productName" "A"}}]}
    {:operation "getAllProducts", :parameters []}]}}
```

Fig. 9 Generated example of creation and delete behaviour

Table 1 Quantitative data from the questioner. The scale of the questions is: 1-Very low, 2-Low, 3-Moderate, 4-High, 5-Very High

Id	Question	Avg
Q1	*"I would rate my experience with REST APIs as"*	2.9
Q2	*"How would you like to rate the readability of the generated examples?"*	3.5
Q3	*"How would you like to rate the quality of the generated examples?"*	4.0
Q4	*"How likely is it that you would keep the generated examples?"*	3.8
Q5	*"To what degree do you think an example generation method could help you better understand an API that you would use?"*	4.2
Q6	*"To what degree do you think an example generation method could help you better understand an API that you develop?"*	4.3
Q7	*"To what degree do you think an example generation method could help you better test an API?"*	4.6
Q8	*"To what degree do you think an example generation method could help you better document an API?"*	3.8

experience in a team with members with more than 20 years of experience might diminish their objective knowledge. 3) Some of the team members work with REST APIs daily, where others might do it monthly, resulting in a feeling of relative lack of experience when compared to other team members. We see this diversity as a positive, as the generated examples have the potential of being useful for users in different roles, with different levels of experience.

The sessions were conducted as online Microsoft Teams meetings. Two of the authors of this paper were present. The first author of this paper took the role of the main moderator, driving the discussion forward. The second author as an observer and complementary moderator, posing further follow-up questions from the discussions.

The procedure of the sessions, inspired by the practical guidelines in (Breen, 2006), was as follows; the main moderator introduced the participants to the agenda, the proposed approach of example generation for REST APIs, and the goal of the session—to collect the participants thoughts about the generated examples. Approval for recording the session was obtained. After an introduction to the topic, examples from all the proposed behavioural properties from Section 3.5 were discussed. The examples were generated by our proposed approach executed on the "features-service", one of the services used in other studies (Kim et al., 2022; Zhang & Arcuri, 2022). This is a real-world service representative of a RESTful API where high coverage can be achieved, which is important since, if a service cannot be covered, we cannot generate examples for it. The service contains 18 endpoints which makes it not too trivial and not too complex to understand. By generating examples using a service the participants are not familiar with, any understanding of the service gained by the participants is from the generated examples presented in the focus group session and not biased by previous exposure to the service. A generated example discussed by the participants is shown in Fig. 10.

In addition to the focus group sessions, we also offered a voluntary questionnaire to perform after the session. This allowed us to capture some quantitative data. 10 participants chose to perform the questionnaire. Table 1 shows data from the questionnaire. The data collected from the questionnaire will be discussed and put into context in the presentation of the qualitative results.

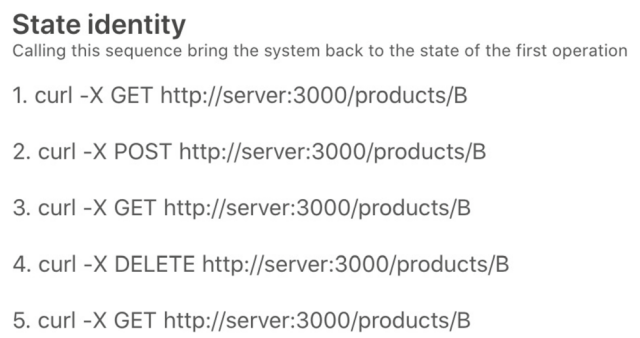


Fig. 10 A generated example shown to the participants during the focus group session

4.2 Results RQ1 - relevance

To analyse the result from the focus group sessions we made a thematic analysis of the transcriptions. Quotes from the participants were coded based on the content of the statement. After coding the statements, larger themes were created to capture the essence of the participants' views. An overview of the themes and codes is shown in Table 2.

Some of the main themes and codes identified relate to *understanding and analysis of the system's behaviour* and different *usage scenarios* for the generated examples. In addition, we identified themes touching on different quality aspects of the generated examples and a *current state of practice* theme. We will present the results of the main themes with supporting quotes from the participants that we include throughout the text in *italics*.

4.2.1 Current state of practice

The participants use examples today, primarily to understand APIs they are using: *"The X API has some documentation with a set of examples and I have read through that and still do"*. Usage scenarios also include understanding how to test APIs; *"from a test perspective we do consume all kinds of documentations including examples"*. Examples are seen as very helpful, and not having them is limiting. This is in line with the current literature—examples are very useful (Robillard, 2009). As one participant commented, from a testing perspective: *"not having examples is a severe limitation"*.

The engineers also recognised that it would be beneficial to create examples of the APIs they develop; *"It would be helpful both for ourselves and everyone that will use our API"*. Putting themselves in the situation of a user helps the design process; *"producing examples will force you to think through the workflow of a user of the API"*. However, producing examples is currently lacking: *"That's not something we do, but we probably should, because I think it will be helpful"*.

In summary, the engineers see API usage examples as helpful and important, but often do not produce them. Thus, our proposed approach to automatically produce examples of behaviours could fill this gap. However, to do so, the generated examples must be relevant to the engineers, which we discuss next.

Table 2 Themes and codes

Theme	Codes
Current State of Practice	Currently only for using other APIs
	We should make examples for our own APIs
	Example usages
	Lack of examples
Using examples for understanding	Dependency on requirements
	Combination says more than individual ones
	"Debugging" the systems through the examples
Desired improvements	Want to see responses
	User experience improvements
	Authorization
	"Relations" not clear
	More clear state changes
These examples are good	Usage of the examples
	Usage in industrial state of practice
	Automation is good
	These examples are helpful to be kept
	Aids in verification
	Aids in understanding
	Aids in test
	Aids in design
	REST best practices
Examples are good	Examples in general are good
	Lack of examples is a limitation
Expressing examples	Curl is good
	Swagger/OpenAPI is good

4.2.2 Understanding and analysing the behaviour of the system

A common theme we observed was that the participants would understand some behaviour of the system based on the examples shown; *"I can infer that by posting a new product it will be added to the list of products. So the POST adds a new product"*. Figure 10 shows an instance of such an example. The engineers could draw specific conclusions from the examples; *"This is how I create a new product in the system"*. Having examples also tells more about the behaviour than what you can infer from only looking at the HTTP method (GET, POST, PUT, DELETE) of the operation; *"[Examples] tells a bit more about operations than just HTTP methods does."*

The participants gained a deeper understanding of the effects of performing a sequence of operations would have on the state of the system. Thus, this understanding goes beyond just the responses of the operations in the example; *"This tells me that this particular POST operation actually does. [change the state]"*, and another engineer noted; *"Another kind of observation I think you can draw from this is that Products seem to be able to *really* create and then *really* delete. Meaning that if I do a DELETE it's gone, there is nothing left of the product that I created"*. As the participant explained in relation to the previous quote, in some systems he is working with entities are not actually deleted, only

marked as such. In addition, that leads to an analysis of the effect on the state of the system as a consequence of the behaviour displayed by the examples, which the example did not explicitly show; *"I would assume that I could create a product B again. And if it didn't show up as a state identity [B4], then I would assume that I cannot create a product anew. I've created it once and now can't do it again"*.

A common theme we observed was that the participants would understand some behaviour of the system from the examples, while also using the examples as a starting point for further analysis of "what if" scenarios. The participants reasoned about the examples seen in the context of how the SUT actually behaves, and in doing so also extended this understanding to what it would mean if an example of the given behaviour was different; *"Nothing else is affected by the GET [operation], like a date timestamp or anything like that. But if it did, I would assume that the system under test is *really* stateful."*, and in addition; *"if we would have an example of a query POST operation that changed the state, that would be very informative."*

Looking at the behaviour examples in aggregate gave the engineers a deeper understanding of how an operation behaves; *"From both these two examples, the first one was response equality [B1] and we got the example that if you do GET twice, it will not change [the response] and this one [B2] says the same thing but from the other perspective, there are no two GET operations that produce different responses. So these two combined definitely tell me that doing these GET operations reads from the system, nothing outside affects the responses of these GET operations"*. Looking at examples of both B1 and B2 for an operation also provided insights about concepts such as idempotency, in addition to how the operation behaves; *"So the first one [operation invocation] could actually insert or create the product zero and the second one [operation invocation] doesn't do anything because it's already created, it just responds back with the same response as the first one. So from the user perspective of the API, it's idempotent"*.

The generated examples show invocations of at least two operation invocations. This was seen as a benefit and a missing piece of specifications; *"I think this is a great example and I think that it's missing when using API specifications, to see the full flow, the chain of calling methods"*.

The behaviours B1–B4 (defined in Section 3.5) were seen as important, *"For me, these are all equally important. They show some properties of a stateful API that is of relevance. Maybe there are more, but these are really important"*. The participants could not think of any other behaviours in addition to B1–B4 that would be useful in the context of generating examples for RESTful APIs; *"I think that it feels like there isn't anymore, these are the minimized examples that exist, or? Is there any more that doesn't fall into these four categories?"*.

The quantitative data collected with regards to understanding support the qualitative analysis in concluding that the engineers find the generated examples relevant in understanding the APIs they develop (Q6 in Tabel 1 with a score of 4.3) and APIs they use (Q5 in Table 1 with a score of 4.2).

The participants understand more about how the system works through the generated examples. The generated examples also stimulate further analysis of the effects of the state of the system. Having multiple examples of different behaviours of the system is helpful for the participants in advancing their understanding.

4.2.3 Usage of the generated examples

In the previous section, we presented the results of the focus group sessions regarding the participants' ability to understand the system under test. In this section, we analyse the results from the theme of *usage*, meaning, in what ways the engineers want to use the generated examples.

In addition to the already discussed usage area of furthering the understanding and triggering of analysis of the behaviour of the system, the participants considered using the examples for verification of behaviours when developing an API, for testing, for documentation, and as an aid when requirements are missing.

The participants thought that the examples would serve as test cases; *"[Example of State Identity] it's a pretty extensive test of the SUT doing all these operations"*, and also as an inspiration for what to test: *"Examples is key to when you don't work with it [the code base] every day and just need to test it somehow"*.

When requirements are missing, generated examples can be a substitute to reason about the behaviour of the system; *"Examples are really good to drive and understand how the system is supposed to work or at least try to find inconsistencies. As long as we don't have any requirements, that's kind of the only way of doing it. So it's really good to have"*.

In the situation where requirements do exist, the generated examples can be used for verification purposes; *"From [the perspective of] developing an API, we have this specification, it should exhibit this kind of behaviour and it should exhibit that kind of behaviour and we could use this [generated examples] to ensure that we follow the specification, using these abstract properties of stateful APIs"*. In addition, the generated examples can help the engineers to reflect on if they have built what they set out to do; *"The example triggers my brain to look at it and investigate if something looks fishy"*. Having a set of generated examples that cover multiple scenarios provides the engineers with trust in that the system does what it is supposed to do; *"From a general perspective also it adds trust on the system when you have such examples. Many times what happens is you don't have any examples of how the API work or there are partial examples not covering all the scenarios. But if you do have examples of these sorts, it builds your trust"*.

Another usage area, mentioned by the engineers, is documentation. The usage of examples for documentation is relevant both for internal and external purposes; *"When you're designing the API, for internal communication and to have a common understanding of how to give examples and describe to each other and in documentation as well"* and another engineer note that; *"I would use them as documentation to create the regression bed for automating the APIs"*. For external documentation, the generated examples can be a complement, *"I think this [the generated examples] is a good complement to other kinds of documentation"*.

The suggested usage areas in the quantitative survey show that usage in testing scenarios is the strongest area (Q7 in Table 1 with a score of 4.6) and that usefulness for documentation is moderate-to-high (scoring 3.8, Q8 in Table 1).

Our results show that the examples generated by our proposed approach are useful to the participants in several different areas, such as test cases, verification of developed behaviour and documentation.

4.2.4 Additional quality aspects of the generated examples

In the overall quantitative quality assessment of the generated examples the participants judged them to be of a high quality (Q3 in Table 1). Quality does not have an exact definition and can mean different things to different participants. In the previous sections, we discussed the aspects of the generated examples helping in understanding the system and how the engineers wanted to use the examples. In this section, we focus on the quality aspects of how the examples are expressed and how the participants wanted to improve them.

The readability of the examples got a quantitative evaluation of 3.5 (Q2 in Table 1), and the qualitative data from the focus group give insights into both the positive aspects of how the examples are expressed and how they can be improved. The examples, as can be seen in Fig. 10, were expressed as Curl[10] commands. This was seen as very positive: *"I think Curl is great. I mean, that's sort of the industry standard of expressing HTTP calls."* and language agnostic: *"everyone who works with their language of choice, they can understand Curl"*. However, several engineers pointed out that the examples should include the status code returned and the actual responses from the executions. Our approach has all this data, but it was not included in the report since it can be quite large and make the report harder to read. However, the engineers would like to have the opportunity to browse this data: *"I would still like to have an opportunity to see the entirety of the responses. But I don't think it should be shown right away, it should be an option to show it."* and also suggested that the differences between the operation executions are shown *"If it would be possible to sort of easily highlight things that change between different responses. For instance, if you have response inequality, what changed? We could show that."*.

A theme that could explain that the usage as documentation has the score 3.8 (lowest of the usage related questions Q5-Q8) and that also might affect the readability of the examples are the values of parameters used in the examples. If the examples would have been produced manually, the engineers would have included parameter values that better fit the domain of the data, *"I think the manual [examples] we've written have better data, the input and output data. We will get at least a little better user-friendly input and output data."*.

One suggested improvement that could potentially increase both the understanding of examples and the readability, is to surround potential state-changing operations with GET-operations. As the participants pointed out, for longer generated examples that include multiple state-changing operations (POSTs for example) it is not always clear which of the operations actually changed the state: *"In the last two examples there are 2 POST operations between the two GETs. It's really not clear what POST operation changed the state.*

[10] https://curl.se/

If we take the first example into consideration, the 1st POST operation seems to change the state, but do we actually know that the second one does?" and to fix this: *"I mean, if there would be a GET between all of the POST then you can [know]".*

The participants judge the quality of the examples to be high. Curl commands are a good way to express REST API examples. To increase readability and understanding, the generated examples should include responses, domain-related values, and more GET operations.

***RQ1**: Based on the evidence presented from the focus groups and survey, we conclude that our proposed approach can generate examples that are **relevant** to engineers; the generated examples help in further the understanding of the API, and is useful for several purposes such as tests and documentation. The suggested improvements are not improvements on the approach, but rather engineering and user-experience work.*

5 Evaluation: test generation

In this section we evaluate the test generation part of the proposed approach. We do so by evaluating the following research questions;

- **RQ2**: How does our behaviour-driven test-generation method compare to test-generation with EvoMaster, based on search-coverage?
- **RQ3**: How do search and execution coverage compare between the methods and what are any limiting factors?

In this paper, we present a novel method of generating tests for an OpenAPI-described REST API by generating examples of how the SUT behaves. This is in contrast to methods focusing on code-coverage and fault-finding. Even though our method is not focused on code-coverage, but rather covering behaviours, we can still use code-coverage as a proxy measure of how much of the behaviour of the SUT we can reach. Therefore, we evaluate our method based on code-coverage, which is the typical measure used in REST-API method evaluations (Kim et al., 2022; Zhang & Arcuri, 2022). In addition, we want to support the claim that using an example generating approach can be done without giving up test-generation possibilities—in addition to producing a source of better understanding the SUT. Since our focus is on behaviour examples, closer to the business logic of the SUT, we can not judge if a specific example is correct or incorrect. The correctness must be assessed based on the requirements of the SUT. However, once an example is deemed as nominal, it can be used for automatic regression testing. For example, if we find an example of where entities with the same "name" property can be created, we can not judge if this is correct or not, only the requirements can tell us. In light of this, we do not measure any fault-finding in our evaluation.

As a comparison with SotA in REST API fuzzing, we compare against EvoMaster. The reason to select EvoMaster is that it has been found in multiple recent studies to be the best on average performing method for REST API fuzzing (Kim et al., 2022; Zhang & Arcuri, 2022). As mentioned, the goals of EvoMaster and our method are different. Thus the comparison does not aim to evaluate which method is "better", but rather to show the potential areas where using a behaviour-based approach can be advantageous.

Since our approach aims at producing examples of behaviours, the outcomes must be thought of in a different way than an approach aiming to generate a test-suite with the goal of code-coverage. Since our approach searches for behaviours, it can also be used to search for examples of behaviours that should not be present. If the intent of the requirements, for example, is that usernames must be unique, the list of examples of that we can POST users with the same username should be empty. Hence, executing the artefact would yield zero coverage of the SUT, which is the expected and correct outcome in this case. On the other hand, if a counter-example is found, indicating a bug, the coverage would increase. Thus, we could see a search-coverage of 100%, while the execution-coverage of the (empty) test-suite would be 0%. This example also highlights that what we propose is not an either/or scenario of using our approach instead of a SotA REST API fuzzer, but rather as a complement, to assess the behaviours of the SUT, and while doing so, generating usage-examples parsable into documentation formats.

This evaluation has several goals. The method needs to cover the code of the SUT to have the potential to assess the behaviours of the SUT. We want to know how much of the SUT the behaviour search can reach (RQ2). Secondly, when behaviours are found, it should be possible to consistently execute the generated test-suite (RQ3). Since we do not use any reset of the SUT during the search or the execution of the resulting test cases, we want to learn what the effect of this strategy is on the execution of the test cases. Creating state-independent test-cases are preferred for successful usage in regression cases.

5.1 Experimental setup

We used EvoMaster version 1.4.[11] We used the documented recommended defaults for black-box testing,[12] with one exception. The default suggests using a rate-limit on requests to the SUT with a maximum of 60 per minute to not cause a denial of service attack. However, since QuickREST has no such rate limit, and to be fair when comparing the outcomes, we removed this limit for EvoMaster. Note that removing this rate limit increases the coverage for EvoMaster, thus the configuration of such a setting would be important to state for any evaluations comparing REST API tools, as it can affect the results between the tools.

The implementation of our method is based on the original QuickREST implementation.[13] In addition to implementing our described approach, we did some engineering additions, such as being able to run the tool as a command-line application, to simplify the execution of the case-studies. Our implementation is openly available.[14]

[11] https://github.com/EMResearch/EvoMaster/releases/tag/v1.4.0
[12] https://github.com/EMResearch/EvoMaster/blob/master/docs/blackbox.md
[13] https://github.com/zclj/replication-packages/tree/master/ICST-2020
[14] https://github.com/zclj/QuickREST

The studies comparing a larger set of REST API fuzzers, execute the tools from 10 min and up to 24 h (Kim et al., 2022) or for one hour (Zhang & Arcuri, 2022). The recommended time for EvoMaster, according to its documentation,[15] ranges from 1 to 24 h.

The time taken for test-generation is of importance in industry (Zhang et al., 2022). This is also very intuitive, the faster the user can get a result, the better. There are several considerations to make concerning time. When executing a fuzzing tool on a continuous integration (CI) server it might be fine for practitioners to wait for the result for hours. But REST API test-generating tools could also be used to give fast feedback when developing or maintaining functionality, as in a test-driven development approach. For example, in the case of our approach, the user can get a generated example of the faulty behaviour, fix the bug, and verify that the behaviour is not generated on the new version. This kind of interactive development would require REST API test-generation tools to be faster, and maybe allow a developer to focus on parts of an API, leaving the deeper fuzzing to the long-running CI-server. The second consideration we do with regard to time is the reset of the SUT. Resetting the SUT with system-level testing, as REST API testing is, can be costly (Zhang et al., 2022), especially in a black-box scenario where the internal state cannot be reset with a library method. Therefore it would be preferable if these tools are robust to searching and executing tests with a changing SUT state. To evaluate with these points in mind, the evaluation of our method aims at producing a result as fast as possible. The time taken was recorded and the same amount of time was given to EvoMaster. The SUTs were restarted before each new complete execution, not during search or execution of the generated test-suite.

The selected SUTs were selected from the EvoMaster Benchmark Suite, included in (Zhang & Arcuri, 2022), that was also included in (Kim et al., 2022). We removed the "restcountries" SUT since the original QuickREST lacks the engineering work to process that OAS.

The SUTs and tools were executed on a virtual Ubuntu 20.04 machine, set up with help of the scripts in (Kim et al., 2022). The host for the virtual machine was a MacBook Pro with 2.9Ghz Intel i9 and 16GB RAM. This setup, is typical for what is used at our industry partner, is representative of an environment a developer might use. Coverage was collected with JaCoCo.[16] Experiments were executed 30 times for each SUT, considering recommendations evaluating random-based algorithms (Arcuri, 2014).

5.2 Results RQ2 - search coverage

Table 3 shows the results from the tools search. Concerning line coverage during search, QuickREST performs slightly higher coverage in two out of eight SUTs; "features-service" (+2.34) and "ocvn-rest" (+2.42), and considerable better in one, "rest-ncs" (+25.7). In four of the eight SUTs, QuickREST perform slightly lower coverage; "languagetool" (−0.51), "proxyprint" (−2.45), "rest-scs" (−1.37), and "scout-api" (−0.83). In one case, EvoMaster produces a considerable higher coverage, "rest-news" (+35.42). In summary, QuickREST, with a behaviour-based method, results in a

[15] https://github.com/EMResearch/EvoMaster
[16] https://www.jacoco.org/jacoco/

Table 3 Achieved code-coverage of the evaluated methods. We report the average line coverage, the [min, max] out of 30 searches (S). In addition, the Mann–Whitney-Wilcoxon U-Test p-values (all being < 0.001) and effect sizes $\hat{A}_{12}$ of EvoMaster vs. QuickREST are included

SUT	Base	EvoMaster BB (S)	QuickREST (S)	$\hat{A}_{12}$	Time(s)
features-service	18.60	57.99 [57.99,57.99]	**60.33 [56.46,73.96]**	0.23	102.8
languagetool	1.32	**2.98 [2.47, 11.87]**	2.47 [2.46, 2.47]	0.76	13.6
ocvn-rest	10.07	25.06 [22.56,26.33]	**27.48 [27.33,27.49]**	0.00	241.6
proxyprint	4.16	**30.86 [30.80,31.37]**	28.41 [28.36,28.50]	1.00	311.0
rest-ncs	4.36	63.62 [62.91,63.64]	**89.32 [80.36,93.82]**	0.00	25.8
rest-news	12.50	**68.06 [68.06,68.06]**	32.64 [32.64,32.64]	1.00	42.4
rest-scs	4.06	**62.81 [62.03,63.39]**	61.44 [61.02,62.03]	0.99	27.5
scout-api	12.23	**18.55 [18.26,18.86]**	17.72 [17.25,18.11]	1.00	48.2

The bold emphasis highlights the best result

similar search coverage in six out of eight SUTs, compared to EvoMaster. The methods have one SUT each with a larger advantage.

Investigating how the search coverage could be increased for each SUT would require an in-depth analysis of the result of each SUT. Such an analysis is out of the scope of this paper. However, we can make some remarks. The "languagetool" case is a known hard case for many REST-API fuzzers (Kim et al., 2022; Zhang & Arcuri, 2022). This SUT is not a very RESTful API, as there is no manipulation of resources. The API only exposes two endpoints, where the functionality of the API is to spell-check a piece of text. Such a non-RESTful API is out of the scope of the proposed properties, we specifically target CRUD-based RESTful APIs, which show in the results.

The second point is the low search coverage, compared to EvoMaster, in the "rest-news" system. An analysis of the search shows that due to engineering deficiency in QuickREST, the tool does not correctly format the input data to POST (creation) endpoints in this API. The result is "Unsupported Media Type" responses (415), which then yield no deeper coverage on these endpoints.

Finally, in the case of "rest-ncs", where QuickREST clearly outperforms EvoMaster, we can note that the API does not have dependencies between operations, i.e., they can be executed in any order, with no dependency on previous operations results. This API property, in combination with that QuickREST moves on to the next behaviour as soon as it has found an example, i.e. this API is "easy" for the behaviour-based search, which seems to yield a good result in a short amount of time. However, whether or not this generalizes to other APIs with a low level of dependencies would require a study with a larger set of such APIs.

***RQ2**: Producing a result as fast as possible, the proposed behaviour-driven test-generation method achieves a search coverage of on average 39.98%. This is comparable to the state-of-the-art fuzzing tool EvoMaster with an on-average search coverage of 41.24%, given the same amount of time.*

Table 4 Achieved code-coverage of the evaluated methods. We report the average line coverage, the [min, max] out of 30 executions (E). In addition, the Mann–Whitney-Wilcoxon U-Test p-values (all being < 0.001) and effect sizes $\hat{A}_{12}$ of EvoMaster vs. QuickREST are included

SUT	Base	EvoMaster BB (E)	QuickREST (E)	$\hat{A}_{12}$
features-service	18.60	35.23 [35.23,35.23]	**48.83 [38.07,69.15]**	0.00
languagetool	1.32	**1.98 [1.91, 2.40]**	1.71 [1.71, 1.71]	1.00
ocvn-rest	10.07	21.74 [19.69,24.55]	**25.66 [25.40,25.75]**	0.00
proxyprint	4.16	12.26 [05.75,**28.03**]	**27.08** [**26.64**,27.72]	0.20
rest-ncs	4.36	**54.13 [52.73,56.36]**	32.36 [32.36,32.36]	1.00
rest-news	12.50	**62.25 [60.42,64.58]**	26.39 [26.39,26.39]	1.00
rest-scs	4.06	**56.70 [53.22,58.64]**	45.76 [45.76,45.76]	1.00
scout-api	12.23	**18.03 [17.55,18.29]**	12.23 [12.23,12.23]	1.00

The bold emphasis highlights the best result

5.3 Results RQ3 - execution coverage

Table 4 shows the execution coverage of the two approaches, with the min and max included. If the goal of an approach is to produce a test-suite with high coverage, then the search and execution coverage should be as close to each other as possible. However, for QuickREST, as mentioned, this is not the goal. Our approach aims at producing examples of behaviours the user might expect, or be surprised by. Hence, it is perfectly fine for the approach to produce a high search coverage, but a low execution coverage, when we do not expect any of the behaviours searched for. In light of this, how do we then evaluate the execution coverage of QuickREST? Even if we do not know the ideal execution coverage for a set of behaviours—without a deep analysis of the actual behaviour of the SUT—we do at least know that the result for the same set of properties on the same SUT should be *consistent*.

QuickREST has a high degree of consistency in seven out of eight of the SUTs, with a small or zero difference in the min and max. The result for one of the SUTs, "features-service", shows a high difference in the average coverage and the min/max results (-10.76, 20.32). This means that for this SUT it is possible to find behaviours with an execution coverage of 69.15% (the max result from Table 4). A reasonable expectation is thus that the approach should find those on each execution. We can also note that this same SUT, is the one that EvoMaster show the highest difference between search and execution coverage (22.76%). For QuickREST the major problem with this SUT, is that examples of the same behaviours are not consistently found. Improvement in the search for behaviours is needed to increase the consistency, while at the same time not significantly increasing the time spent in search. EvoMaster shows a different problem. While EvoMaster is very consistent in both the search and execution coverage for "features-service", it produces test-cases that are dependent on the exact state of the SUT as it was when searching. Hence, when these tests are executed after a reset of the SUT, or at a later time with a different state, the test fails. For example, a test tries to delete an entity with a specific id, without making sure such an entity is first created. Such a test fails when the entity is not present, thus the execution coverage is lower.

***RQ3**: The main limiting factor for QuickRESTs behaviour-driven method to increase the consistency in execution coverage is to decrease the variability in the search result of the SUT's behaviour.*

6 Discussion

While researchers have made great improvements in the code covered and the faults found with REST API fuzzing, there are plenty of open challenges. Challenges include; handling state in the SUT (Zhang et al., 2022), using testing criteria more related to business logic (Zhang et al., 2022), the time taken while searching for test cases (Zhang et al., 2022), the readability of generated tests (Zhang et al., 2022), the optimization of computational resources (Martin-Lopez et al., 2022), stronger support for stateful testing (Kim et al., 2022), and finding dependencies between operations (Kim et al., 2022). In addition, several state-of-the-art methods do not produce an executable artefact when the fuzzing process is completed (Zhang & Arcuri, 2022). This greatly reduces the possibility of fast regression testing for practitioners, as re-doing the complete search/fuzzing process is wasteful. Also, we argue, there is a missed opportunity in the search process; to improve the understanding of the SUT, and how the SUT relates to certain *behaviours*.

With the evaluations of the proposed approach, we have shown that an example-generation approach can address several of the open challenges. The generated examples are deemed relevant by practitioners, increasing their understanding of the SUT. In addition, the practitioners find several different uses for the generated examples, such as a source of documentation and verification of best practices. Generating an artefact with multiple usage areas means that an example-generating approach has the potential to provide a higher yield on the time spent searching for test cases. In addition, the generated examples are on the higher abstraction level of behaviours, compared to test cases generated to further a coverage metric. This brings them closer to the level of abstraction of business logic.

One big difference between our approach compared to approaches that produce an executable artefact with specific values in the test-cases, is that the behaviour-based examples are more state-independent. This is something we could observe with the comparison to EvoMaster. Examples generated from the "features-service" case-study highlight this difference. Consider the example generated by EvoMaster in Fig. 11. We can see in this example a "get" of "/products" (L4) and the exact specific items expected (L10-11). It is very unlikely that this test will ever succeed after its generation since it is highly specific. To make this test pass on a restarted SUT, the test-suite would need to include the creation of these entities—which it does not. If we instead look at what our method produces, in Fig. 8, we can observe that the behaviour of the same "get"-operation is exercised in a more state-independent fashion. It makes no difference in this test how many other entities are in the system, the test asserts the behaviour that the result of the "get"-operation is affected by the "post"-operation, which is the essence of the "get"-operation. A similar situation can be seen in Fig. 12. In this test, there is a "delete" on the entity "/products/7" (L4) which is expected to be successful (L6). However, once again, the success of this test relies on that there exists an entity with id "7", which is not created by this test, or

```
@Test
public void test_1() throws Exception {
  given().accept("application/json")
          .get(baseUrlOfSut + "/products")
          .then()
          .statusCode(200)
          .assertThat()
          .contentType("application/json")
          .body("size()", equalTo(5))
          .body("", hasItems("ELEARNING_SITE",
          "NRIuqSIDyR", "H7uPs1", "YPofE4WoBC", "nJojT"));
}
```

Fig. 11 State-dependant "GET" test-case generated by EvoMaster

the test-suite. In contrast, the behaviour-based method for deletion produces the example in Fig. 9. In this example an entity is first created to then be deleted, putting the state of the "get" operation back to where we started. This test is self-contained and highlights a behaviour, it also works on a restarted system.

Tests created as examples of behaviours are thus less dependent on the state of the system since they are expressed on a higher level of abstraction—the exact values of entities are not evaluated, but rather the overall behaviour. While our method currently produces tests that do not depend on the state of the SUT, future work is to make sure this invariant holds for a complete set of behaviours, i.e., values should not be reused over a complete set of behaviours. If you do want generated tests to be very specific on the values returned, the EvoMaster approach is a good complement—*if* the test-suite would create the required conditions for successful execution.

Another strength of the proposed behaviour-based approach is that the user gets control of the search on a higher abstraction level. The user can choose what behaviours to search for—some behaviours might be irrelevant to the type of SUT the user has—thus time and energy will only be spent towards that goal. All the proposed behaviours in this paper can be individually used. This is in contrast to approaches with a more opaque search method with less high-level control for the user.

The final strength of the proposed approach we highlight is the potential for the generated examples to be used for more than test-cases. As we have shown, the generated examples are self-contained. The examples can be post-processed into other formats since the output is a data-structure—not source code. Behavioural examples have been shown to help users in understanding a system (Gerdes et al., 2018). This is an area that could be explored further, for example, to study how users prefer to have generated usage examples

```
@Test
public void test_6() throws Exception {
  given().accept("*/*")
          .delete(baseUrlOfSut + "/products/7")
          .then()
          .statusCode(204)
          .assertThat()
          .body(isEmptyOrNullString());
    }
}
```

Fig. 12 State-dependant "DELETE" test-case generated by EvoMaster

of RESTful APIs presented—based on our focus groups, Curl is well received, but there might be better alternatives not yet investigated.

A weakness of a behaviour-based approach is that if the behaviours do not correspond to the domain of the SUT, they will not produce a good search result. Since the proposed first set of behaviour-properties in this paper are targeted towards RESTful APIs, APIs such as "language-tool" that are not very RESTful are not applicable for our approach with the current proposed behaviours. Indeed, the method is only as strong as the match between the domain of the SUT and the behaviours used.

Finally, regarding the usage of the proposed approach. We foresee the approach to be useful in a development process where software engineers can use it in an interactive fashion to understand if the API under development is providing the expected behaviours. The engineer can generate examples, analyze the result, make changes to the software, and generate new examples. When the engineer is satisfied with the examples and the behaviour of the software, the generated examples can be saved and executed on new versions of the software, as regression tests. Quality assurance engineers can also benefit from the approach, for example when integrating and understanding APIs created by different engineering teams, or as support when understanding undocumented APIs. The usefulness of the approach in these cases is supported by the results of our evaluation of the relevance of the generated examples in Section 4. With these use cases in mind, we can see a combination of approaches as beneficial. The approach proposed in this paper might be applied in the active development cycle of an engineer—make a change, assess the result—, while an approach focusing on code coverage and fault finding, such as EvoMaster, can complement when the change to the software is complete—for example, running on a continuous integration server.

Our proposed behaviour-based test-generation approach produces test-cases that are on a higher abstraction level. They are state-independent, allow the user to have control of the selection of behaviours, and have the potential for extensions of its output to areas such as documentation. But the method is limited to the relation between the domain and the specific behaviours. In order to increase the search result for non-RESTful APIs, additional behaviours are needed.

7 Related work

REST API fuzzing and test-generation have been a thriving research area in recent years, with many methods proposed (Arcuri, 2019; Atlidakis et al., 2019, 2020a, b; Corradini et al., 2022; Ed-douibi et al., 2018; Godefroid et al., 2020a; Karlsson et al., 2020; Laranjeiro et al., 2021; Martin-Lopez et al., 2021; Segura et al., 2018; Stallenberg et al., 2021; Viglianisi et al., 2020; Wu et al., 2022). However, the main test oracles used

focus on finding crashes and conformance between the SUTs API and the OAS. There are exceptions, such as a focus on security properties (Atlidakis et al., 2020b) or breaking changes between API-versions (Godefroid et al., 2020b). In addition, metamorphic testing has been used to find bugs regarding common REST API relations, manually defined (Segura et al., 2018). To the best of our knowledge, no other work has focused on common behavioural properties, automatically, related to business logic, as we do.

In the area of REST API fuzzing, there are two other proposed methods based on property-based testing (Claessen & Hughes, 2000), QuickREST (Karlsson et al., 2020) and Schemathesis (Hatfield-Dodds & Dygalo, 2021). The properties in these methods, as in the general case for other methods, focus on finding API crashes, conformance to specification, and in the case of Schemathesis, the possibility for performance checks, and conformance to HTTP semantics (such as content type). As described, our method is not primarily targeting fuzzing, but behaviours expected of a CRUD-based REST APIs.

EvoMaster (Arcuri, 2019) have been extended with resource-based templates (Zhang et al., 2021) to improve in performing multiple operations on the same resource, to achieve higher code coverage of the tests. RESTTESTGEN also base their dependencies on resource relations (Viglianisi et al., 2020). We use a type-based graph, which is less sensitive to APIs not fully following REST API guidelines, since we do not depend on the defined resources in the specification. In addition, to be able to perform multiple operations on the same resource, we leverage the composition of the proposed behaviours, where to be conformant with a behaviour, the same resource needs to be used.

There exist several methods for generating examples to support the understanding of software (Barnaby et al., 2020; Buse & Weimer, 2012; Gerdes et al., 2018; Gu et al., 2016, 2019; Holmes et al., 2006; Karlsson et al., 2024; Kim et al., 2009; Mar et al., 2011; Martin & Guo, 2022; Mittal & Paris, 1994; Montandon et al., 2013; Moreno et al., 2015). However, most of these approaches rely on white-box information (Barnaby et al., 2020; Buse & Weimer, 2012; Gu et al., 2016, 2019; Holmes et al., 2006; Kim et al., 2009; Mar et al., 2011; Martin & Guo, 2022; Montandon et al., 2013; Moreno et al., 2015). Two approaches which do not require white-box information to generate examples are Gerdes et al. (2018) and Karlsson et al. (2024). Gerdes et al. proposed a black-box example generating approach which requires a formal specification of the behaviour of the SUT to be able to generate relevant examples (Gerdes et al., 2018). The limitation of requiring a formal specification was removed with the approach proposed by Karlsson et al. (2024). The approach by Karlsson et al. is able to generate relevant examples in a black-box fashion by using generally defined behaviours—as meta-properties. In this paper, we build on this approach by specialising it in the context of REST APIs. In addition, a specialised context makes the approach more relevant to evaluate and compare to other approaches in that same context—REST APIs test generation in our case.

In summary, our main difference from previous approaches is to drive test generation with a focus on finding behaviours of the SUT. This produces examples of the behaviours the SUT conforms to, usable to both support understanding and as a source of test generation.

8 Threats to validity

In this section, we discuss the relevant to this paper threats to validity based on Wohlin et al. (2012).

Internal validity—are the results caused by our approach. We have developed a prototype of the proposed method which was used in both evaluations. The prototype is available as open source[17]

External validity—how general the results are. We have used 8 different SUTs in our experiments, which are representative of different kinds of REST APIs. The SUTs are used in benchmarks in previous publications (Arcuri, 2019; Kim et al., 2022; Zhang et al., 2021; Zhang & Arcuri, 2022). A deeper description of the SUTs can be found in Zhang and Arcuri (2022). The deeper description can be helpful to understand how the results might generalize to a new kind of REST API not included in the benchmark, but with similar characteristics. To allow the reader to judge if the results generalize to other development teams, we provide information on the composition of the groups and discuss their experience levels and roles.

Randomness is a threat to the construct and conclusion validity of our experiments. To mitigate this, we follow standard recommendations on performing experiments with randomized algorithms (Arcuri, 2014).

9 Conclusions

Today, REST APIs are a common way for services to provide functionality, both internally, as part of a larger system, and to external clients. This popularity has been reflected in the research community, with many different test generation and fuzzing approaches proposed. However, there are still challenges in this area.

In this paper, we have mainly focused on bringing test generation for REST APIs closer to the actual business logic. We have done so by proposing a behaviour-based approach of generating examples and an initial set of CRUD-based behavioural-properties. Our evaluation shows that a behaviour-based approach can provide similar coverage to a SotA search-based method, while at the same time producing less state dependent test cases, due to the focus on behaviours. In addition, the approach provides generated examples valuable to practitioners.

We see this work as a starting point that can be further extended with more behaviours, covering a larger domain of REST APIs—more than CRUD-based APIs. Further, the method can be extended to be more consistent in the search results provided.

Searching for and generating test cases can be time-consuming and require many resources. To be respectful of practitioners' time and resources, we should aim for approaches that make the most of the time spent. In addition, time is an essential factor to enable interactive workflows for practitioners. With this work, we have introduced an approach that automates the creation of relevant examples of API behaviours that can serve as (i) a source for engineers to understand the system and (ii) a source of automated test case generation. We have shown that our approach provides these additional benefits while matching—in a given time span—test coverage of SOTA fuzzing approaches. Users do not

[17] https://github.com/zclj/QuickREST

have to choose to "only" get test cases as output. We can also provide them with means of better understanding their systems—an important part of creating high-quality reliable software systems.

Acknowledgements This work is supported by ABB, the industrial postgraduate school Automation Region Research Academy (ARRAY) funded by The Knowledge Foundation.

Author contributions All authors contributed to the writing of the main manuscript and the development of the presented approach. The first draft of the manuscript was written by Stefan Karlsson; then, all other coauthors updated and commented on previous versions of the manuscript. Stefan Karlsson implemented the approach for use in the evaluations. The focus group evaluation was performed, including data analysis, by Stefan Karlsson and Robbert Jongeling. The test generation evaluation was performed by Stefan Karlsson. All authors reviewed the manuscript and approved the final version.

Funding Open access funding provided by Mälardalen University. This work is funded by ABB, and The Knowledge Foundation (KKS).

Data availability The implementation used is available at https://github.com/zclj/QuickREST. The generated examples shown in the focus group sessions are available at https://figshare.com/s/d038fc8822aa5963ff0c. Any further data clarifications are available from the corresponding author upon request.

Declarations

Competing interest The authors declare no competing interests.

Open Access This article is licensed under a Creative Commons Attribution 4.0 International License, which permits use, sharing, adaptation, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if changes were made. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit http://creativecommons.org/licenses/by/4.0/.

References

Aghajani, E., Nagy, C., Linares-Vásquez, M., Moreno, L., Bavota, G., Lanza, M., & Shepherd, D. C. (2020). Software documentation: the practitioners' perspective. In *Proceedings of the ACM/IEEE 42nd International Conference on Software Engineering*, ICSE '20 (p. 590–601). ACM. Retrieved from https://doi.org/10.1145/3377811.3380405

Arcuri, A. (2019). RESTful API automated test case generation with EvoMaster (vol. 28). https://doi.org/10.1145/3293455

Arcuri, A., & Briand, L. (2014). *A Hitchhiker's guide to statistical tests for assessing randomized algorithms in software engineering., 24*, 219–250. https://doi.org/10.1002/stvr.1486

Atlidakis, V., Geambasu, R., Godefroid, P., Polishchuk, M., & Ray, B. (2020a). *Pythia: Grammar-based fuzzing of REST APIs with coverage-guided feedback and learning-based mutations*. https://doi.org/10.48550/ARXIV.2005.11498

Atlidakis, V., Godefroid, P., & Polishchuk, M. (2019). RESTler: Stateful REST API fuzzing. In *2019 IEEE/ACM 41st International Conference on Software Engineering (ICSE)* (pp. 748–758). https://doi.org/10.1109/ICSE.2019.00083

Atlidakis, V., Godefroid, P., & Polishchuk, M. (2020b). Checking security properties of cloud service REST APIs. In *2020 IEEE 13th International Conference on Software Testing, Validation and Verification (ICST)* (pp. 387–397). https://doi.org/10.1109/ICST46399.2020.00046

Barnaby, C., Sen, K., Zhang, T., Glassman, E., & Chandra, S. (2020). Exempla Gratis (E.G.): Code examples for free. In *Proceedings of the 28th ACM Joint Meeting on European Software Engineering*

Conference and Symposium on the Foundations of Software Engineering, ESEC/FSE 2020 (pp. 1353–1364).

Breen, R. L. (2006). *A practical guide to focus-group research* (vol. 30, pp. 463–475). Routledge. https://doi.org/10.1080/03098260600927575

Buse, R. P. L., & Weimer, W. (2012). Synthesizing API usage examples. In *2012 34th International Conference on Software Engineering (ICSE)* (pp. 782–792). https://doi.org/10.1109/ICSE.2012.6227140

Claessen, K., & Hughes, J. (2000). *QuickCheck: a lightweight tool for random testing of Haskell programs* (vol. 35, pp. 268–279). https://doi.org/10.1145/357766.351266

Corradini, D., Zampieri, A., Pasqua, M., Viglianisi, E., Dallago, M., & Ceccato, M. (2022). *Automated black-box testing of nominal and error scenarios in RESTful APIs* (vol. 32, p. e1808). https://doi.org/10.1002/stvr.1808

Ed-douibi, H., Cánovas Izquierdo, J. L., & Cabot, J. (2018). Automatic generation of test cases for REST APIs: a specification-based approach. In *2018 IEEE 22nd International Enterprise Distributed Object Computing Conference (EDOC)* (pp. 181–190). https://doi.org/10.1109/EDOC.2018.00031

Fielding, R. (2000). *Architectural styles and the design of network-based software architectures*. Irvine, US: University of California.

Fowler, S. J. (2016). *Production-ready microservices*. O'Reilly.

Gerdes, A., Hughes, J., Smallbone, N., Hanenberg, S., Ivarsson, S., & Wang, M. (2018). Understanding formal specifications through good examples. In *Proceedings of the 17th ACM SIGPLAN International Workshop on Erlang*, Erlang 2018 (pp. 13–24). https://doi.org/10.1145/3239332.3242763

Godefroid, P., Huang, B. -Y., & Polishchuk, M. (2020a). Intelligent REST API data fuzzing. In *Proceedings of the 28th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, ESEC/FSE 2020 (pp. 725–736). https://doi.org/10.1145/3368089.3409719

Godefroid, P., Lehmann, D., & Polishchuk, M. (2020b). *Differential regression testing for REST APIs* (pp. 312–323). https://doi.org/10.1145/3395363.3397374

Golmohammadi, A., Zhang, M., & Arcuri, A. (2023). *Testing restful apis: a survey*. https://doi.org/10.1145/3617175

Gu, X., Zhang, H., & Kim, S. (2019). CodeKernel: a graph kernel based approach to the selection of API usage examples. In *2019 34th IEEE/ACM International Conference on Automated Software Engineering (ASE)* (pp. 590–601). https://doi.org/10.1109/ASE.2019.00061

Gu, X., Zhang, H., Zhang, D., & Kim, S. (2016). Deep api learning. In *Proceedings of the 2016 24th ACM SIGSOFT International Symposium on Foundations of Software Engineering*, FSE 2016 (pp. 631–642) New York, USA: Association for Computing Machinery. https://doi.org/10.1145/2950290.2950334

Hatfield-Dodds, Z. & Dygalo, D. (2021). Deriving semantics-aware fuzzers from web API schemas. arXiv. https://doi.org/10.48550/ARXIV.2112.10328

Holmes, R., Walker, R. J., & Murphy, G. C. (2006). *Approximate structural context matching: an approach to recommend relevant examples* (vol. 32, pp. 952–970). https://doi.org/10.1109/TSE.2006.117

Karlsson, S., Čaušević, A., & Sundmark, D. (2020). QuickREST: Property-based test generation of OpenAPI-described RESTful APIs. In *2020 IEEE 13th International Conference on Software Testing, Validation and Verification (ICST)* (pp. 131–141). https://doi.org/10.1109/ICST46399.2020.00023

Karlsson, S., Hughes, J., Jongeling, R., Čaušević, A., & Sundmark, D. (2024). Exploring API behaviours through generated examples. *Software Quality Journal*. https://doi.org/10.1007/s11219-024-09668-2

Kim, J., Lee, S., Hwang, S.-w., & Kim, S. (2009). Adding examples into java documents. In *2009 IEEE/ACM International Conference on Automated Software Engineering* (pp. 540–544). https://doi.org/10.1109/ASE.2009.39

Kim, M., Xin, Q., Sinha, S., & Orso, A. (2022). Automated test generation for REST APIs: No time to rest yet. In *Proceedings of the 31st ACM SIGSOFT International Symposium on Software Testing and Analysis*, ISSTA 2022 (pp. 289–301). https://doi.org/10.1145/3533767.3534401

Kontio, J., Lehtola, L., & Bragge, J. (2004). Using the focus group method in software engineering: Obtaining practitioner and user experiences. In *Proceedings. 2004 International Symposium on Empirical Software Engineering, 2004. ISESE '04.* (pp. 271–280). https://doi.org/10.1109/ISESE.2004.1334914

Laranjeiro, N., Agnelo, J., & Bernardino, J. (2021). *A black box tool for robustness testing of REST services., 9*, 24738–24754. https://doi.org/10.1109/ACCESS.2021.3056505

Mar, L. W., Wu, Y. -C., & Jiau, H. C. (2011). Recommending proper API code examples for documentation purpose. In *2011 18th Asia-Pacific Software Engineering Conference* (pp. 331–338). https://doi.org/10.1109/APSEC.2011.18

Martin, J., & Guo, J. L. C. (2022). Deep api learning revisited. In *Proceedings of the 30th IEEE/ACM International Conference on Program Comprehension*. ICPC '22 (pp. 321–330), New York, USA: Association for Computing Machinery. https://doi.org/10.1145/3524610.3527872

Martin-Lopez, A., Segura, S., & Ruiz-Cortés, A. (2021). RESTest: Automated black-box testing of RESTful web APIs. In *Proceedings of the 30th ACM SIGSOFT International Symposium on Software Testing and Analysis* ISSTA 2021 (pp. 682–685). https://doi.org/10.1145/3460319.3469082

Martin-Lopez, A., Segura, S., & Ruiz-Cortés, A. (2022). Testing of RESTful APIs: Promises and challenges. In *30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE '22)*.

McLellan, S., Roesler, A., Tempest, J., & Spinuzzi, C. (1998). *Building more usable APIs* (vol. 15, pp. 78–86). https://doi.org/10.1109/52.676963

Mittal, V. O., & Paris, C. (1994). Generating examples for use in tutorial explanations: Using a subsumption based classifier. In *In Proceedings of the 11th European Conference on Artificial Intelligence*.

Montandon, J. E., Borges, H., Felix, D., & Valente, M. T. (2013). Documenting APIs with examples: Lessons learned with the APIMiner platform. In *2013 20th Working Conference on Reverse Engineering (WCRE)* (pp. 401–408). https://doi.org/10.1109/WCRE.2013.6671315

Moreno, L., Bavota, G., Di Penta, M., Oliveto, R., & Marcus, A. (2015). How can I use this method? In *2015 IEEE/ACM 37th IEEE International Conference on Software Engineering* (vol. 1, pp. 880–890). https://doi.org/10.1109/ICSE.2015.98

Novick, D. G., & Ward, K. (2006). What users say they want in documentation. In *Proceedings of the 24th Annual ACM International Conference on Design of Communication* SIGDOC '06 (pp. 84–91). https://doi.org/10.1145/1166324.1166346

Nykaza, J., Messinger, R., Boehme, F., Norman, C. L., Mace, M., & Gordon, M. (2002). What programmers really want: Results of a needs assessment for SDK documentation. In *Proceedings of the 20th Annual International Conference on Computer Documentation* SIGDOC '02 (pp. 133–141). https://doi.org/10.1145/584955.584976

Piccioni, M., Furia, C. A., & Meyer, B. (2013). An empirical study of API usability. In *2013 ACM / IEEE International Symposium on Empirical Software Engineering and Measurement* (pp. 5–14). https://doi.org/10.1109/ESEM.2013.14

Robillard, M. P. (2009). *What makes APIs hard to learn? Answers from developers* (vol. 26, pp. 27–34). https://doi.org/10.1109/MS.2009.193

Robillard, M. P., & DeLine, R. (2011). *A field study of API learning obstacles* (vol. 16, pp. 703–732). https://doi.org/10.1007/s10664-010-9150-8

Segura, S., Parejo, J. A., Troya, J., & Ruiz-Cortés, A. (2018). Metamorphic testing of RESTful web APIs. In *Proceedings of the 40th International Conference on Software Engineering* ICSE '18 (p. 882). https://doi.org/10.1145/3180155.3182528

Serbout, S., & Pautasso, C. (2024). APIstic: a large collection of OpenAPI metrics. In *2024 21st IEEE/ACM International Conference on Mining Software Repositories (MSR 2024)*.

Serbout, S., Lauro, F. D., & Pautasso, C. (2022). Web APIs structures and data models analysis. In *2022 IEEE 19th International Conference on Software Architecture Companion (ICSA-C)* (pp. 84–91). https://doi.org/10.1109/ICSA-C54293.2022.00059

Shull, F., Lanubile, F., & Basili, V. (2000). *Investigating reading techniques for object-oriented framework learning* (vol. 26, pp. 1101–1118). https://doi.org/10.1109/32.881720

Sohan, S. M., Maurer, F., Anslow, C., & Robillard, M. P. (2017). A study of the effectiveness of usage examples in rest api documentation. In *2017 IEEE Symposium on Visual Languages and Human-Centric Computing (VL/HCC)* (pp. 53–61). https://doi.org/10.1109/VLHCC.2017.8103450

Stallenberg, D., Olsthoorn, M., & Panichella, A. (2021). Improving test case generation for REST APIs through hierarchical clustering. In *2021 36th IEEE/ACM International Conference on Automated Software Engineering (ASE)* (pp. 117–128). https://doi.org/10.1109/ASE51524.2021.9678586

Viglianisi, E., Dallago, M., & Ceccato, M. (2020). RESTTESTGEN: Automated black-box testing of RESTful APIs. In *2020 IEEE 13th International Conference on Software Testing, Validation and Verification (ICST)* (pp. 142–152). https://doi.org/10.1109/ICST46399.2020.00024

Wohlin, C., Runeson, P., Höst, M., Ohlsson, M. C., Regnell, B., & Wesslén, A. (2012). *Experimentation in software engineering*. Springer Science & Business Media.

Wu, H., Xu, L., Niu, X., & Nie, C. (2022). Combinatorial testing of RESTful APIs. In *44th International Conference on Software Engineering (ICSE '22)*. https://doi.org/10.1145/3510003.3510151

Zhang, M., & Arcuri, A. (2022). *Open problems in fuzzing RESTful APIs: a comparison of tools*. https://doi.org/10.48550/ARXIV.2205.05325

Zhang, M., Arcuri, A., Li, Y., Xue, K., Wang, Z., Huo, J., & Huang, W. (2022). *Fuzzing microservices in industry: Experience of applying EvoMaster at Meituan*. https://doi.org/10.48550/ARXIV.2208.03988

Zhang, M., Marculescu, B., & Arcuri, A. (2021). *Resource and dependency based test case generation for RESTful Web services*. https://doi.org/10.1007/s10664-020-09937-1

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.